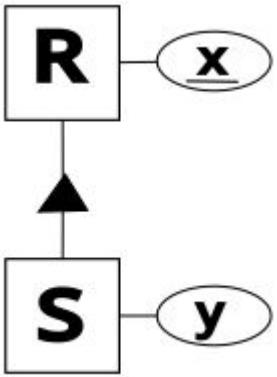
CSC 370: Databases Improving Conceptual Designs

Sean Chester • University of Victoria

Which table matches the subset?




- ```
  CREATE TABLE `S`(
    `y` INT PRIMARY KEY
  );
  ```

- ```
  CREATE TABLE `S`(
    `x` INT PRIMARY KEY,
    `y` INT,
    FOREIGN KEY(`x`)
      REFERENCES `R`(`x`)
  );
  ```

- ```
  CREATE TABLE `S`(
    `x` INT,
    `y` INT PRIMARY KEY,
    FOREIGN KEY(`y`)
      REFERENCES `R`(`x`)
  );
  ```

- ```
  CREATE TABLE `S`(
    `x` INT,
    `y` INT PRIMARY KEY,
    FOREIGN KEY(`x`)
      REFERENCES `R`(`x`)
  );
  ```

Today's Goal:

You can improve the quality of a conceptual schema and objectively argue for the merit of your improvements

Outline

1. Review examples from previous lesson
2. Define qualities of a "good" conceptual schema
3. Work through some case studies to identify possible schema improvements
4. Summary / Exercise

Last class...

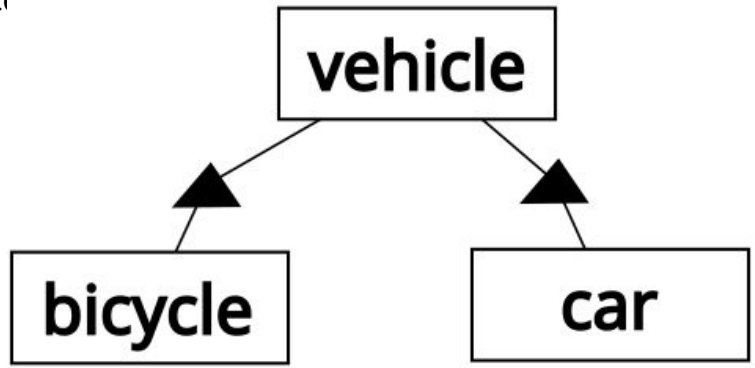
Generalisation

Generalisation defines one concept as being a more specific type of another concept (which you've seen as *inheritance* in object-oriented programming)

- The concept *vehicle* is more general than the concepts of *bicycle* and *car* or a *electric* and *combustion*.
- The concept of an *employee* is more general than the concepts of *manager*, *clerk*, and *professor*

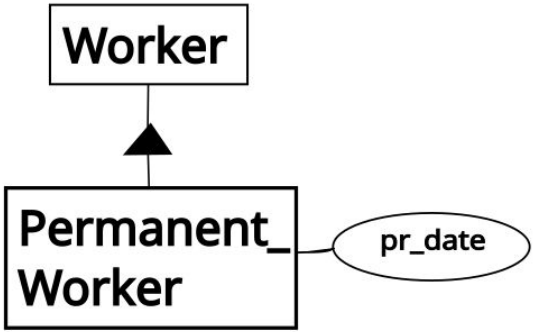
Importantly, with generalisation, all properties of the *parent* concept are inherited by all of the children

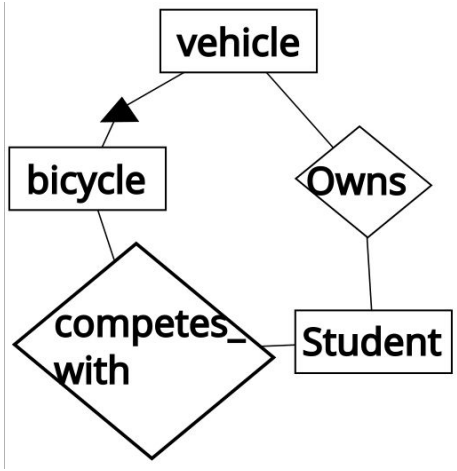
We represent subsets with a black arrow pointing to the parent concept



Why use subsets?

- This arises specifically when there is a subset of an entity set to which additional attributes or an additional relationship applies
  - In the ERD below, we see that for permanent (visa) workers, we also want to track the date they received permanent residence (pr_date), which is not relevant to other workers
  - In the ERD to the right, creating a subset allows us to construct a more expressive relationship that can only involve certain types of vehicles



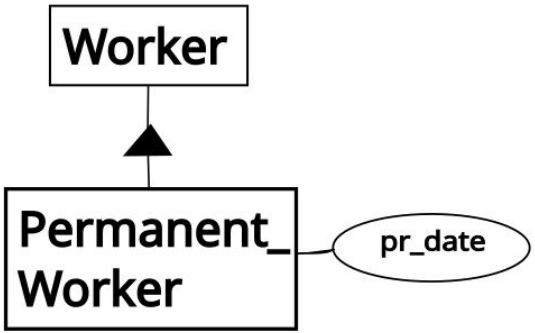


Subsets as tables

- A subset is easy to recognise in SQL DDL because the primary key is also a foreign key

```
CREATE TABLE `Worker`(`id` int PRIMARY KEY);

CREATE TABLE `PermanentWorker`(
  `id` int PRIMARY KEY,
  `pr_date` DATETIME NOT NULL,
  FOREIGN KEY(`id`) REFERENCES `Worker`(`id`)
);
```



Weak Entity Sets

- A *strong* entity set E, like those on the previous slide, is one in which the identifiers are a subset of the attributes of E
- A *weak* entity set E, instead, has as part of its identifier attributes that come from some other entity set E'
- We illustrate a weak entity set with a double rectangle and a double diamond on the relevant relationship

What are the identifiers in the schema given below?

- Genus has a single rectangle. Therefore it is a strong entity set with {*name*} as the identifier
- Species has a double rectangle and is *supported by* relationship *Belongs-to*. Therefore it is a weak entity set with {*name*, *Genus.name*} as its identifier.

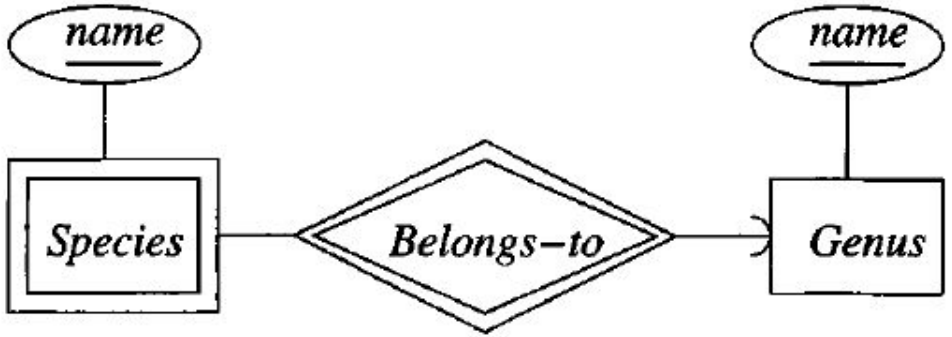


Figure 4.21: Another weak entity set, for species

Source: Garcia-Molina et al. (2009), §4.4

Weak entity sets as tables

- A weak entity set is easy to recognise in SQL DDL because *part of* the primary key is also a foreign key

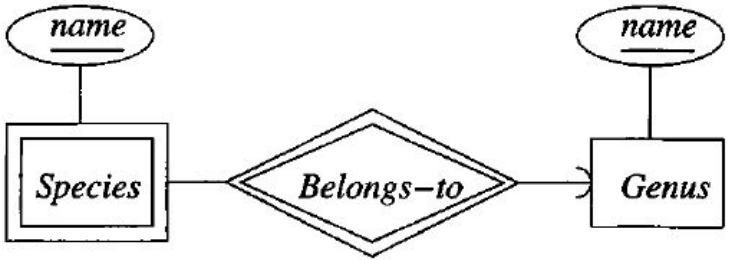


Figure 4.21: Another weak entity set, for species

```
CREATE TABLE `Genus`(
  `name` varchar(60) PRIMARY KEY
);

CREATE TABLE `Species`(
  `genus` varchar(60),
  `name` varchar(60),
  PRIMARY KEY(`genus`, `name`),
  FOREIGN KEY(`genus`)
    REFERENCES `Genus`(`name`)
);
```

Qualities of a good conceptual design

Design Criteria

What aspects would we expect of a *good* conceptual design?
(I have regrouped these somewhat compared to the text)

Completeness: it represents all features of the application domain. In principle, this can be checked by comparing all requirements of the application domain and checking that they are captured somewhere in the conceptual schema.

Correctness: it properly uses the concepts of the ER model. Syntactically correct when the right notions are used (e.g., no generalisation of relationships); It is semantically correct if the notions are used correctly. Typical errors include:

- Using an attribute instead of an entity set (or vice versa)
- Forgetting a generalisation or implied inheritance
- A relationship with two few/many entity sets
- Forgetting an identifier or min/max specification

Source: Batini et al (1992)., §6.1

Design Criteria

What aspects would we expect of a *good* conceptual design?
(I have regrouped these somewhat compared to the text)

Minimality: every aspect of the schema appears in the requirements and is represented in the schema only once. It is minimal if no element can be deleted without violating the requirements or losing some information.

Expressiveness: it represents requirements in a natural way and can be easily understood without explanation.

Readability: it respects certain aesthetic criteria that make the diagram graceful, say by:

- Drawing the diagram on a grid with horizontal/vertical lines
- Using symmetry, few line crossings, and few line bends
- Parents should be drawn above children in the diagram

Source: Batini et al (1992)., §6.1

Design Criteria

What aspects would we expect of a *good* conceptual design?
(I have regrouped these somewhat compared to the text)

Self-Explanation: a large number of annotations can be expressed with the schema itself, without, say, text annotations.

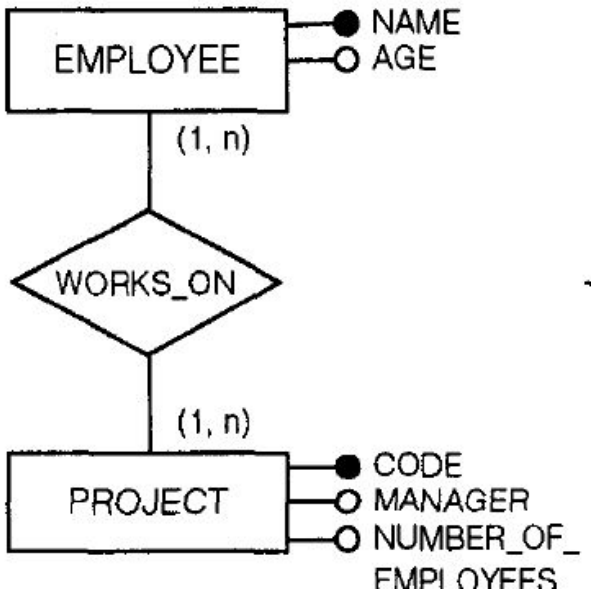
Extensibility: it is easily changed to adapt to changing requirements, e.g., because it is decomposed into pieces (modules or views) so that changes are localised

Normality: it would, if represented as a relational database, fit into standard normal forms like Boyce-Codd, 3NF, or 4NF.
(We'll discuss this when we get there!)

Source: Batini et al (1992)., §6.1

Example 1: Employee Project Database

What is the biggest issue with this design and how would you improve it?

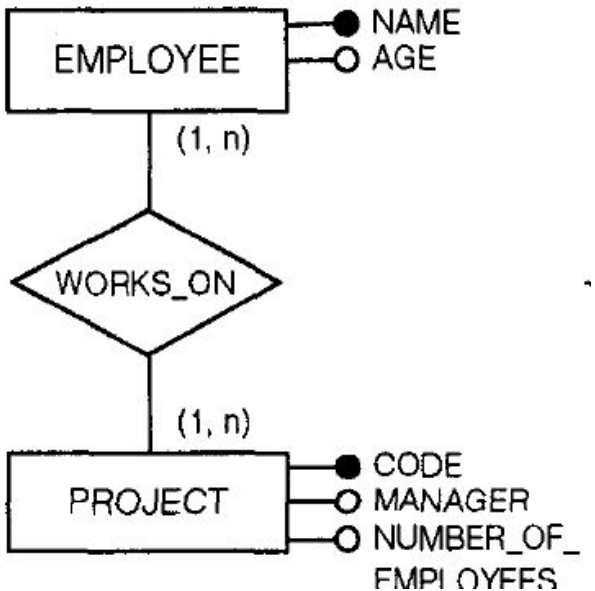


Modeling example; not activity

Source: Batini et al (1992)., §6.1

Example 1: Employee Project Database

What is the biggest issue with this design and how would you improve it?



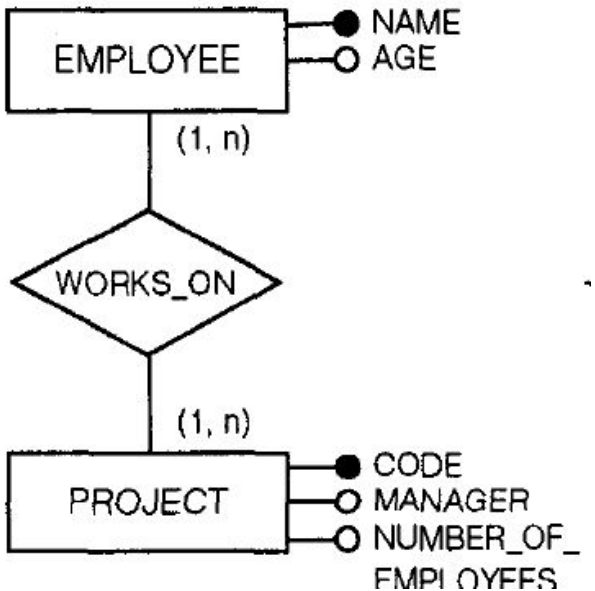
Completeness

- *All features are incorporated*
- In this example, we do not have a list of features; so, we cannot evaluate completeness. Therefore, it is reasonable to assume there is no error of this sort (though to be really precise, we do not have a basis for evaluating it).

Source: Batini et al (1992)., §6.1

Example 1: Employee Project Database

What is the biggest issue with this design and how would you improve it?



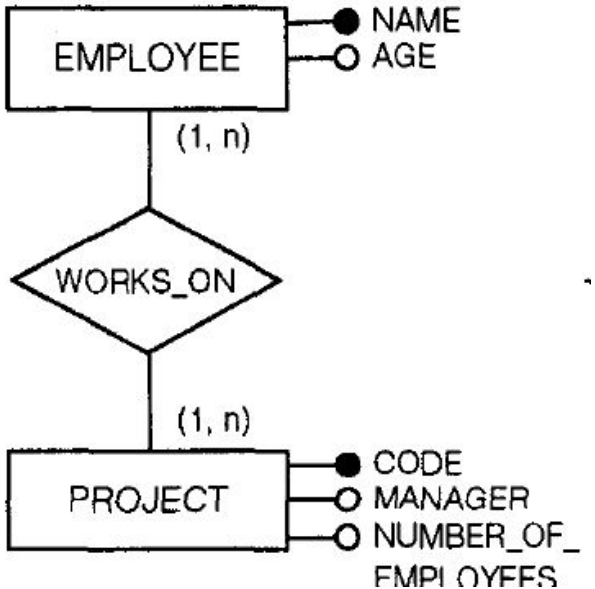
Correctness

- *The model is used with appropriate syntax*
- In this example, the authors use a different "flavour" of entity-relationship diagrams, but the schema is correct with respect to that flavour. (In this course, you should use the syntax from the Garcia-Molina text.)

Source: Batini et al (1992)., §6.1

Example 1: Employee Project Database

What is the biggest issue with this design and how would you improve it?



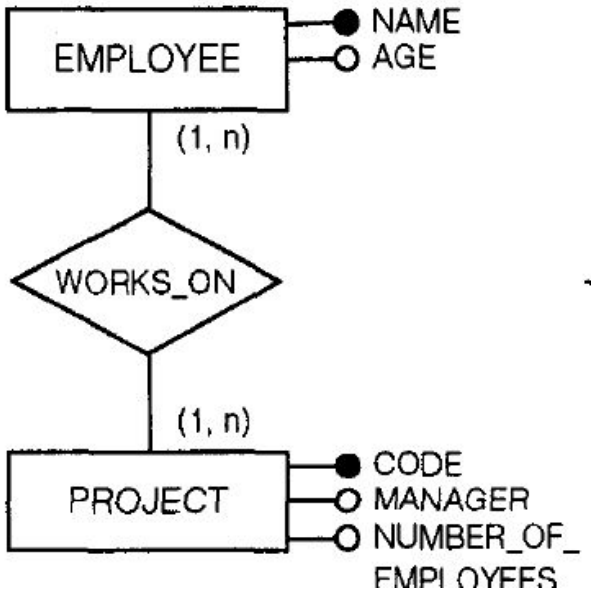
Minimality

- *Every aspect of the schema appears exactly once*
- In this example, minimality is not respected! This is the key error.
- The attribute *number_of_employees* is redundant because it is already captured by the number of tuples in the *Works_on* relationship
- There may also be redundancy with respect to the *manager* attribute, though it isn't clear from this example, because that is probably captured in another relationship called *Manages*

Source: Batini et al (1992)., §6.1

Example 1: Employee Project Database

What is the biggest issue with this design and how would you improve it?



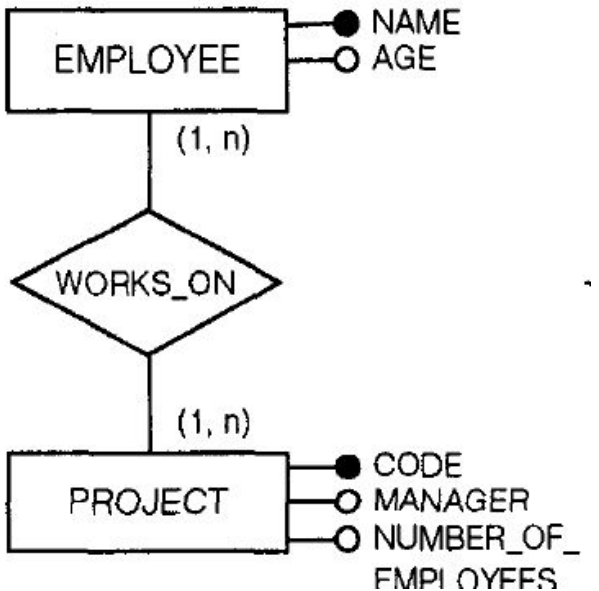
Expressiveness

- *The schema is natural and idiomatic*
- In this example, expressiveness is reasonably good
- At a quick glance, we understand everything going on
- Exceptions to expressiveness (Are managers employees? Does the number of employees include something other than this relationship?) are better described by the error in minimality

Source: Batini et al (1992)., §6.1

Example 1: Employee Project Database

What is the biggest issue with this design and how would you improve it?



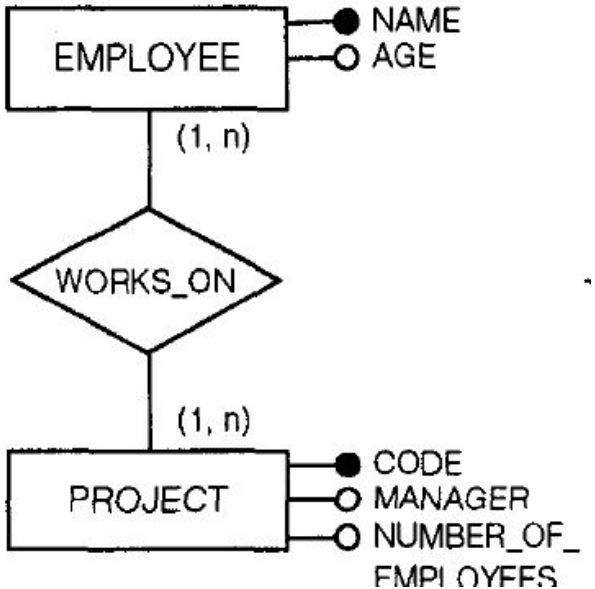
Readability

- *The schema is legible and aesthetically clean*
- In this example, readability is excellent
- Symmetry and consistent placement of attributes, identifiers make the schema easy to read

Source: Batini et al (1992)., §6.1

Example 1: Employee Project Database

What is the biggest issue with this design and how would you improve it?



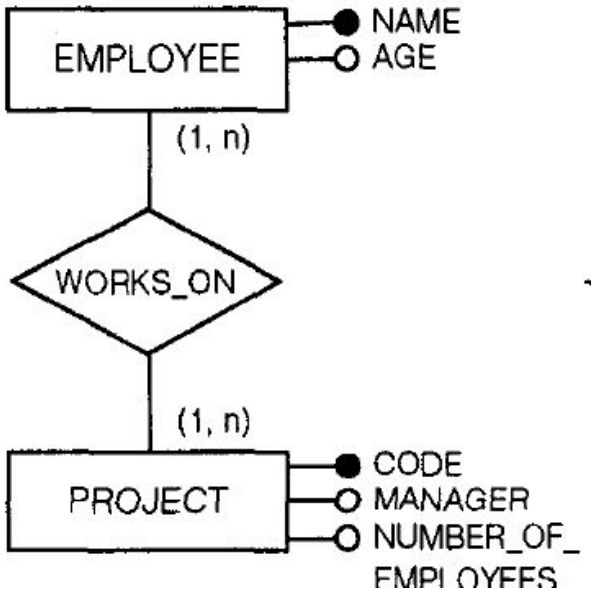
Self-Explanation

- *The schema can be understood w/o annotations*
- In this example, self-explanation is excellent
- Entity set, attribute, and relationship names are well chosen
- The multiplicity on the relationship communicates the key information
- Exceptions to self-explanation (Are managers employees? Does the number of employees include something other than this relationship?) are better described by the error in minimality

Source: Batini et al (1992)., §6.1

Example 1: Employee Project Database

What is the biggest issue with this design and how would you improve it?



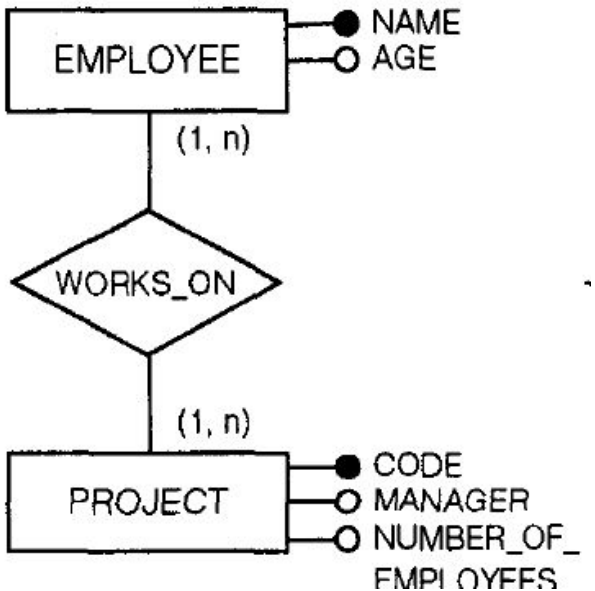
Extensibility

- *The schema can easily adapt to changing req's*
- In this example, extensibility is mediocre
- Some attributes are hard to reverse (e.g., converting age to birthdate)
- Some challenges with minimality might lead to data inconsistencies
- On the other hand, one could easily expand the *Project* and *Employee* entity sets into larger sub-schemata

Source: Batini et al (1992)., §6.1

Example 1: Employee Project Database

What is the biggest issue with this design and how would you improve it?



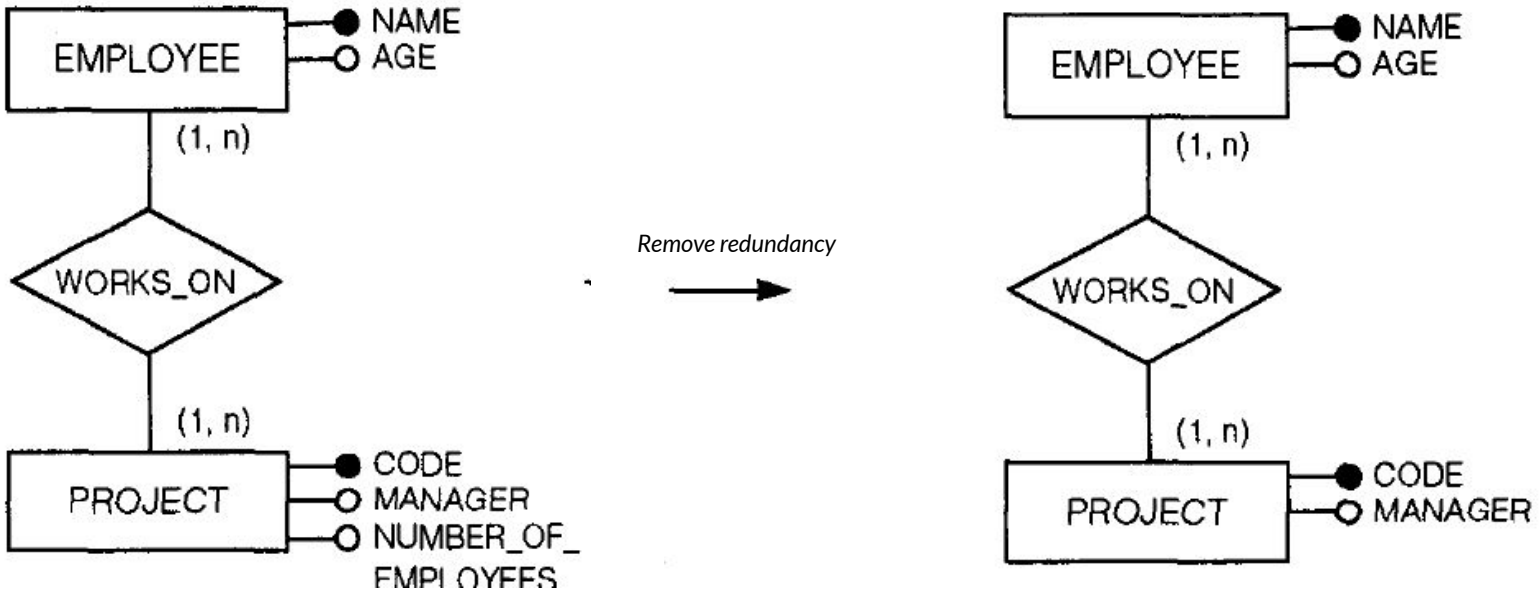
Normality

- *The schema corresponds to a normal form in the relational data model*
- In this example, normality is fine
- For each identifier attribute, there is a functional dependency to each non-identifier attribute in the same entity set
- There are no one-to-many or one-to-one relationships

Source: Batini et al (1992)., §6.1

Example 1: Employee Project Database

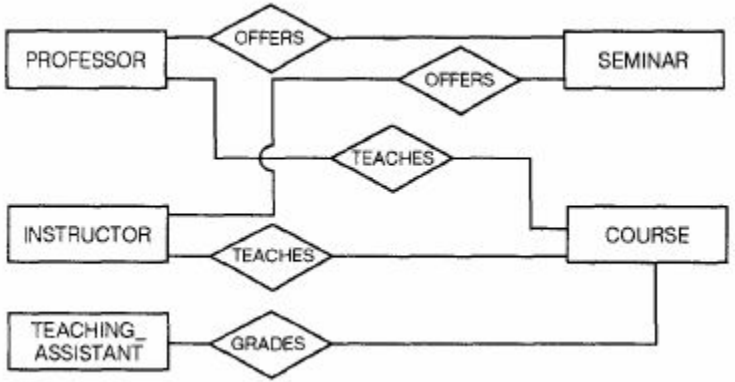
What is the biggest issue with this design and how would you improve it?



Source: Batini et al (1992)., §6.1

Example 2: University Course Database

What is the biggest issue with this design and how would you improve it?



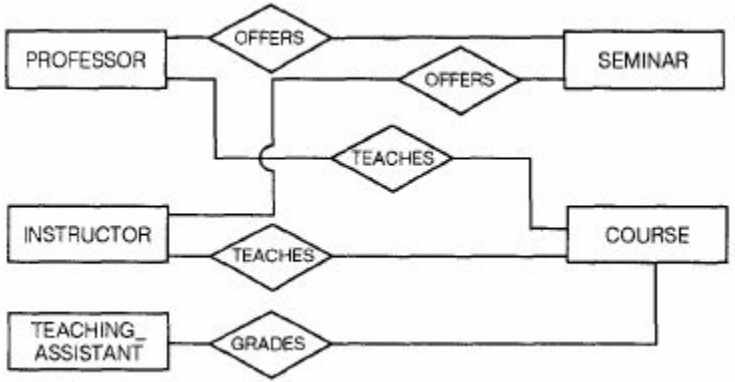
- Completeness?
- Correctness?
- Minimality?
- Expressiveness?
- Readability?
- Self-explanation?
- Extensibility?
- Normality?

Time Allotted: 120 seconds

Source: Batini et al (1992)., §6.1

Example 2: University Course Database

What is the biggest issue with this design and how would you improve it?



Expressiveness

- *The schema is natural and idiomatic*
- It is hard to understand why there are two sets of relationships that have the same name as each other
- There should be a more natural way to represent this idea without the repetition
- (This type of repetition is *often* a “design smell”)

Time Allotted: 120 seconds

Source: Batini et al (1992)., §6.1

Example 2: University Course Database

What is the biggest issue with this design and how would you improve it?

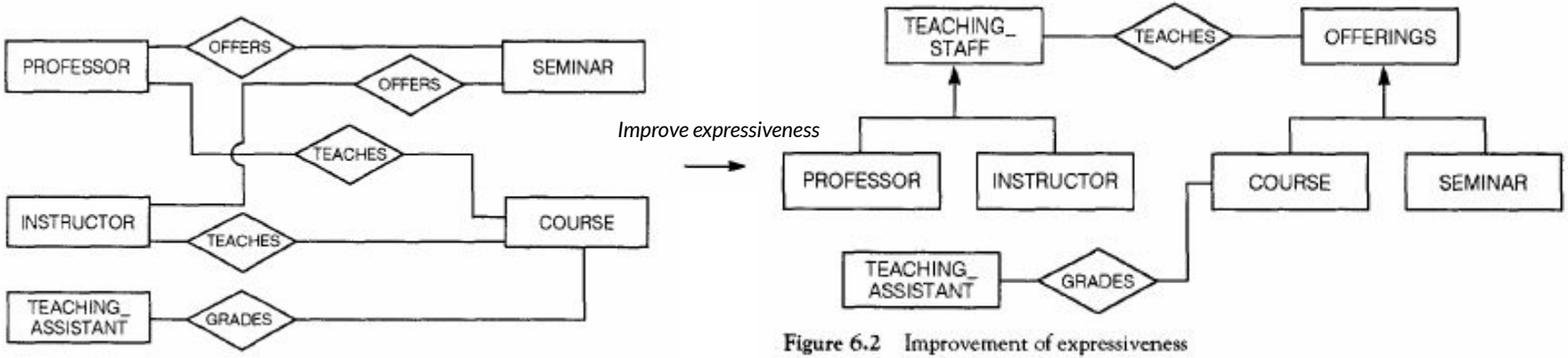
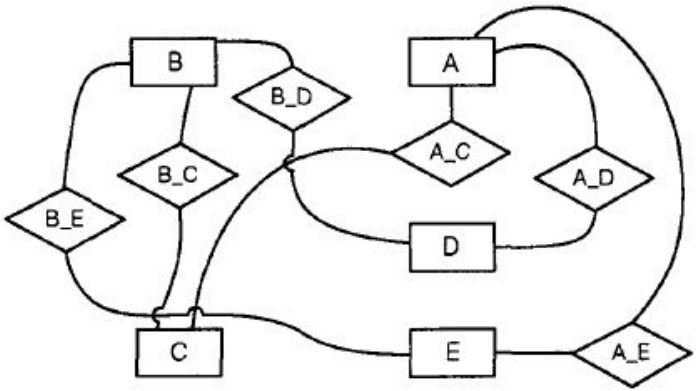


Figure 6.2 Improvement of expressiveness

Source: Batini et al (1992)., §6.1

Example 3: Generic Database

What is the biggest issue with this design and how would you improve it?



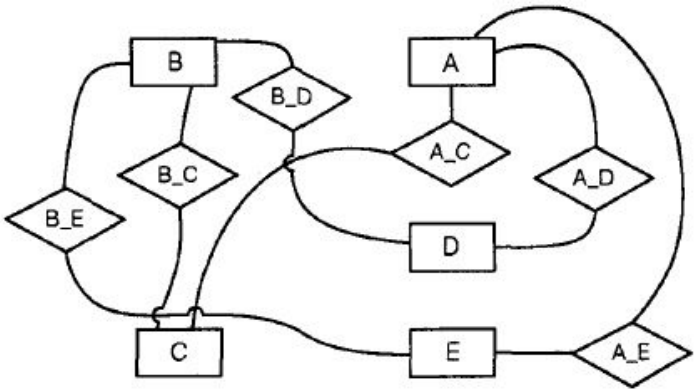
- Completeness?
- Correctness?
- Minimality?
- Expressiveness?
- Readability?
- Self-explanation?
- Extensibility?
- Normality?

Time Allotted: 60 seconds

Source: Batini et al (1992)., §6.1

Example 3: Generic Database

What is the biggest issue with this design and how would you improve it?



Readability

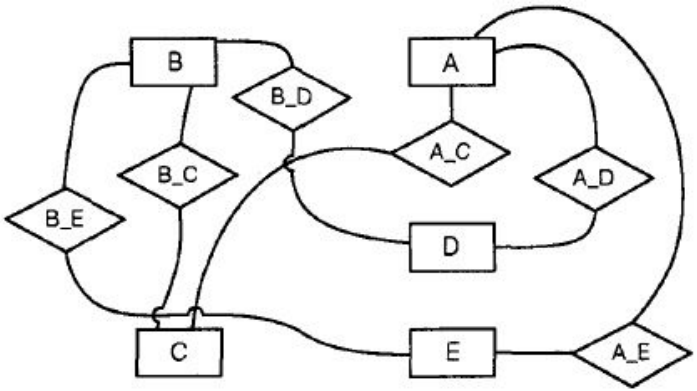
- *The schema is legible and aesthetically clean*
- It is hard to see exactly which entity sets are related to each other without relying on the naming convention used here
- Several curved lines are distracting
- It isn't clear at a glance how many relationships each entity set is involved in
- (This would never be generated by a tool)

Time Allotted: 60 seconds

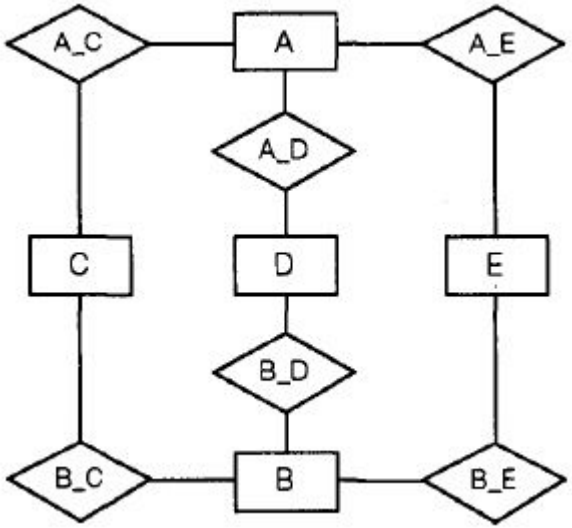
Source: Batini et al (1992)., §6.1

Example 3: Generic Database

What is the biggest issue with this design and how would you improve it?



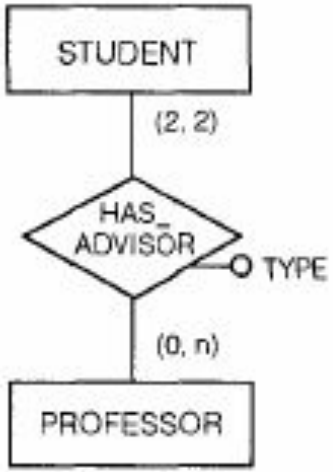
Improve readability



Source: Batini et al (1992)., §6.1

Example 4: Student Advisor Database

What is the biggest issue with this design and how would you improve it?



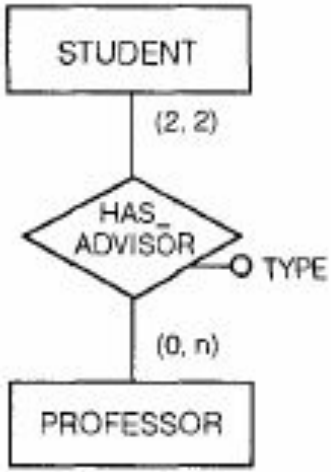
- Completeness?
- Correctness?
- Minimality?
- Expressiveness?
- Readability?
- Self-explanation?
- Extensibility?
- Normality?

Time Allotted: 120 seconds

Source: Batini et al (1992)., §6.1

Example 4: Student Advisor Database

What is the biggest issue with this design and how would you improve it?



Self-explanation

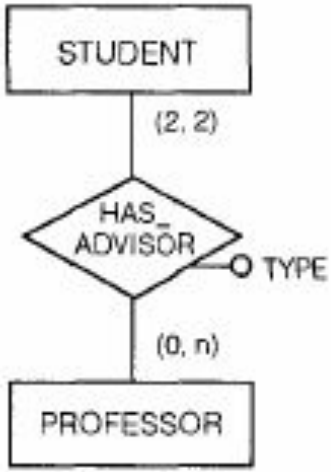
- *The schema can be understood w/o annotations*
- This schema introduces a lot of questions; in fact, without extraneous information, it is hard to even know how to improve it
- Why must a student have two advisors?
- Are those advisors necessarily distinct?
- What is the *type* attribute and why does it exist on the relationship?
- Can *type* be the same for both advisors?

Time Allotted: 120 seconds

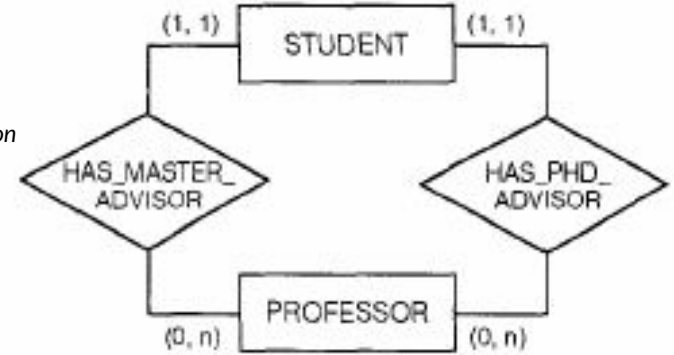
Source: Batini et al (1992)., §6.1

Example 4: Student Advisor Database

What is the biggest issue with this design and how would you improve it?



Improve self-explanation



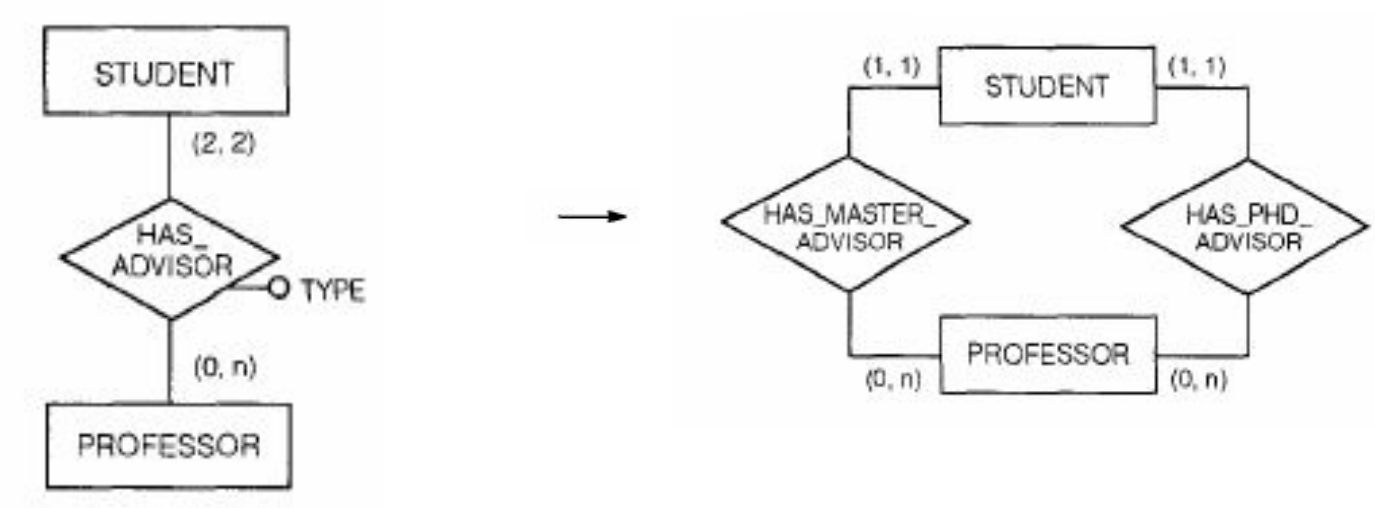
Source: Batini et al (1992)., §6.1

Schema transformations

Schema Transformations

A *schema transformation* produces a new schema S2 from an input schema S1. We saw several examples on the previous slides, e.g.,

Schema S1 **Transformation** **Schema S2**



Source: Batini et al (1992)., §6.2

Schema Transformations

A *schema transformation* produces a new schema S2 from an input schema S1. We classify them based on the *information content*:

- **Information-preserving**: The information content of the schema is not changed by the transformation.

- **Information-changing**:
  - *Augmenting*: The information content of the resulting schema is greater than that of the input schema
  - *Reducing*: The information content of the resulting schema is less than that of the input schema.
  - *Incomparable*: otherwise.

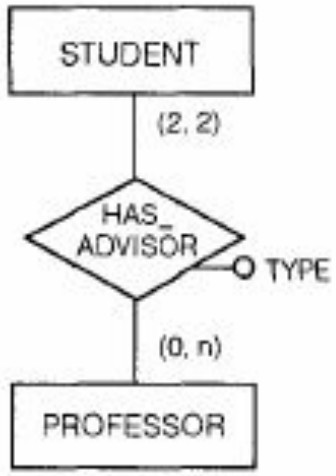
We usually used augmenting transformations in design or with completeness errors.
We use reducing transformations to eliminate superfluous information
We use information-preserving ones to improve expressiveness/readability/etc.

Source: Batini et al (1992)., §6.2

Schema Transformations

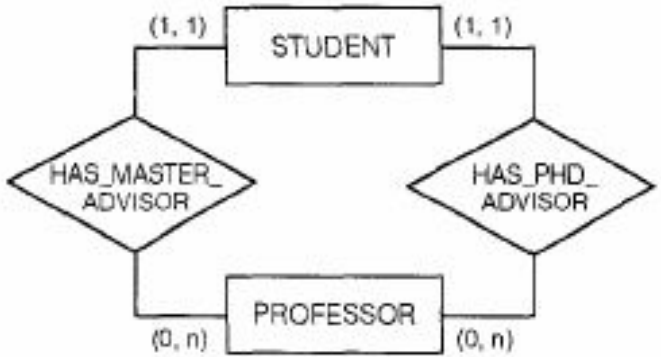
This example is *information-preserving*. The "new" details were in the previous schema, just represented implicitly.

Schema S1



Information-Preserving Transformation

Schema S2



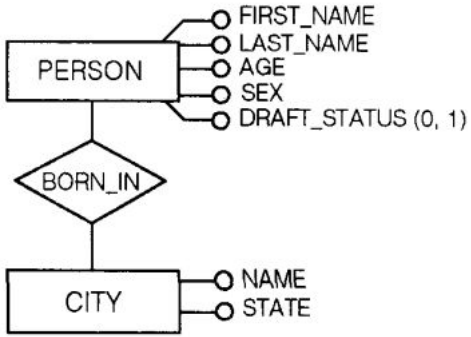
Source: Batini et al (1992)., §6.2

Schema Transformations

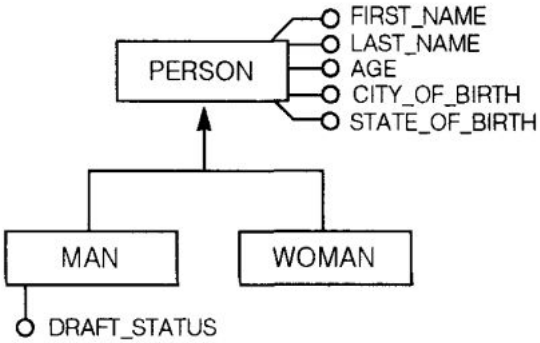
- **Information-preserving**: The information content of the schema is not changed by the transformation.

- **Information-changing**:
  - *Augmenting*: The info content of the resulting schema is greater than that of the input schema
  - *Reducing*: The information content of the resulting schema is less than that of the input schema.
  - *Incomparable*: otherwise.

What type of transformation does the second schema to the right represent?



(a) First schema



(b) Second schema

Time Allotted: 60 seconds

Source: Batini et al (1992)., §6.2

Common Transformations

Achieving Minimality: Cycles

Redundancy exists when one relationship R_1 between two entities has the same information content as a path of relationships $R_2, R_3, \ldots, R_k$ connecting exactly the same pairs of entity instances as R_1

Is has_tel_2 the employee's manager's phone number?

Iff so, it is redundant.

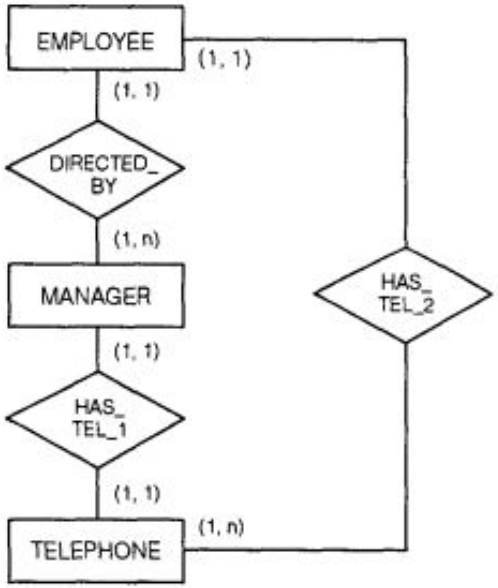
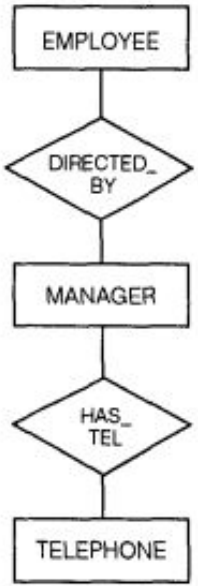


Figure 6.6 Minimality depends on the meaning

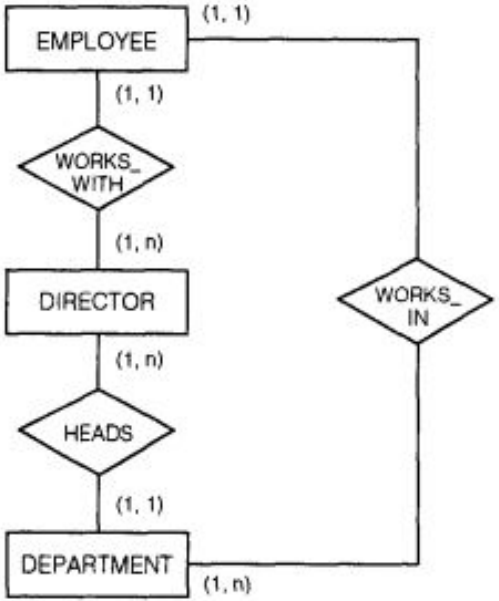


Source: Batini et al (1992)., §6.3

Achieving Minimality: Cycles

Redundancy exists when one relationship R_1 between two entities has the same information content as a path of relationships $R_2, R_3, \ldots, R_k$ connecting exactly the same pairs of entity instances as R_1

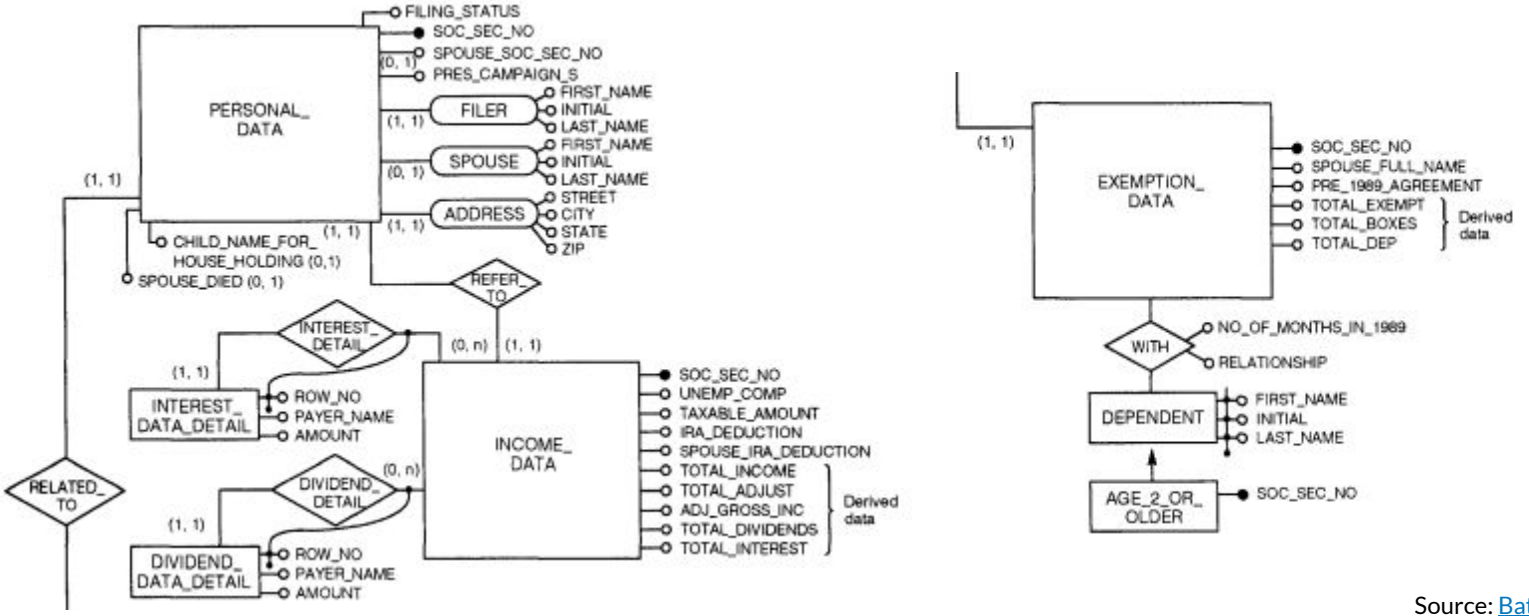
Here, works_in *is clearly implied by the existing relationship from Employee to Director and Director to Department*



Source: Batini et al (1992)., §6.3

Achieving Minimality: Derived Attributes

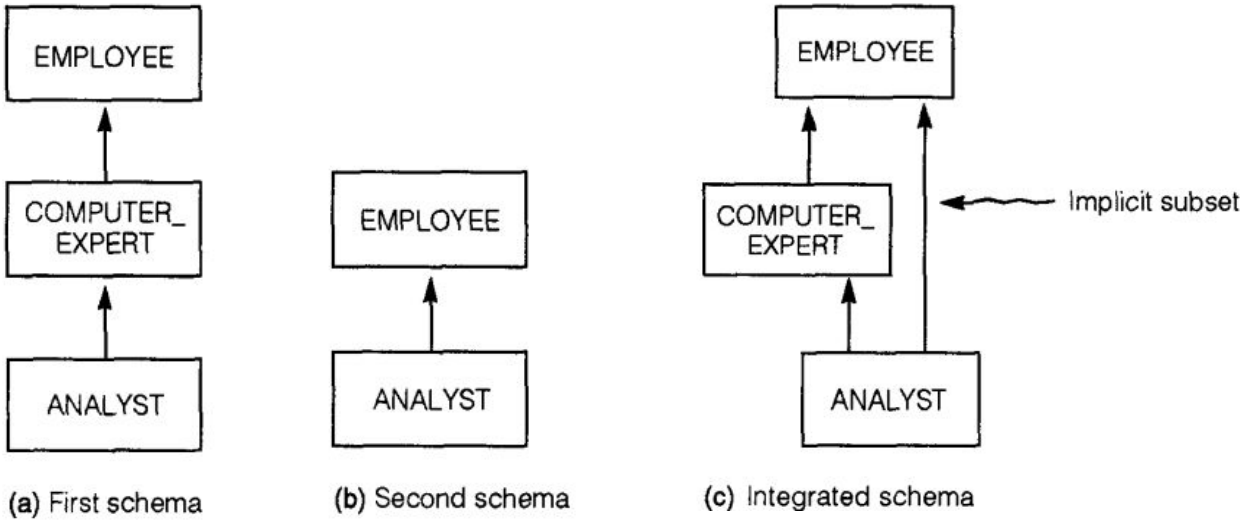
Redundancy may be due to the existence of an algorithm for computing the values of derived data from the other data; hence derived data can be omitted from a minimal ER schema



Source: Batini et al (1992)., §4.3, 6.3

Achieving Minimality: Implicit Subsets

Some of the subsets might be derived from other subsets present in the schema. In the example below, the first and second schemata are combined to produce the third, which is clearly not minimal because the inheritance of Analyst from Employee is already implied transitively through Computer_Expert



Source: Batini et al (1992)., §6.3

Achieving Expressiveness: Dangling Subs

It may happen that the designer creates a generalization in the process of assigning different properties to entities in the hierarchy. If, at the end of the design process, the sub-entities are not distinguished by any specific property, they can be reduced to the super-entity.

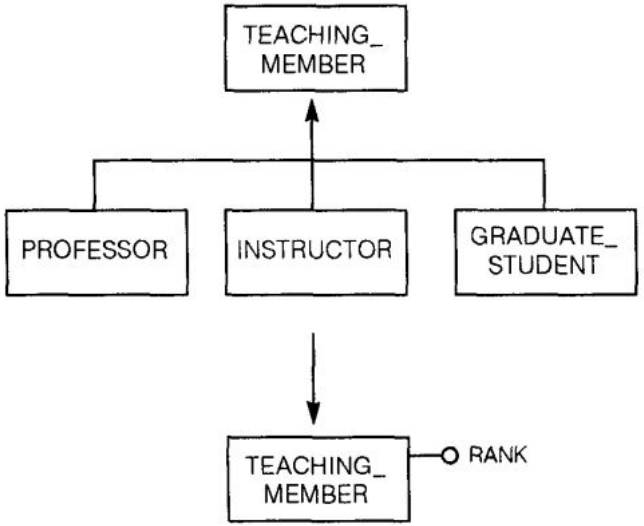


Figure 6.9 Elimination of a nonrepresentative generalization hierarchy

Here, none of the sub-entities have differentiating attributes or relationships. There is thus no value in this complexity.

Source: Batini et al (1992)., §6.4

Achieving Expressiveness: Dangling Entities

We regard an entity E as *dangling* if it has few (possibly one) attributes A, and one connection to another entity (the principal entity).

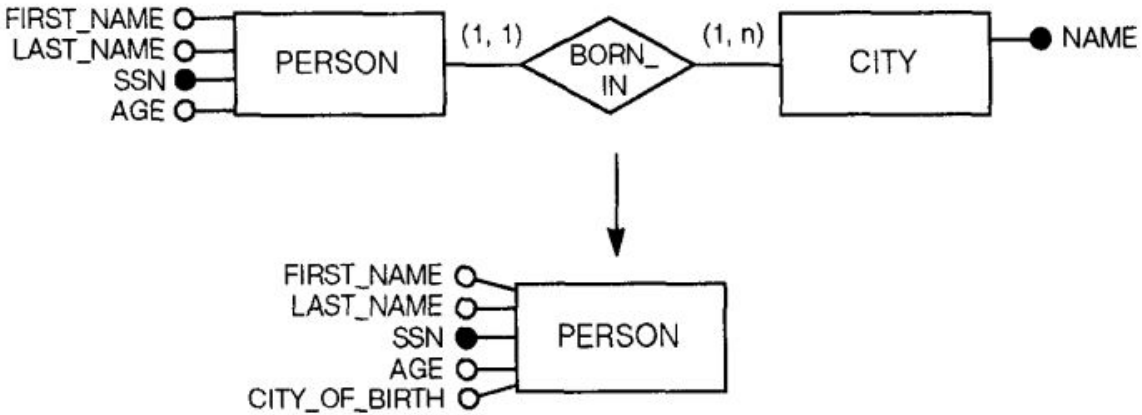


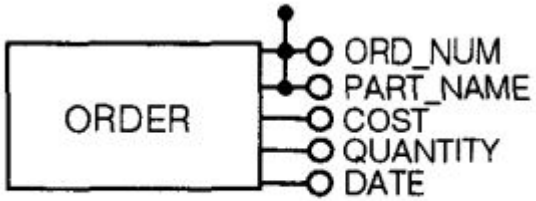
Figure 6.10 Elimination of a dangling entity

Here, we have a simpler design by modelling city *as an attribute; it has no non-identifier attributes and no other relationships.*

Source: Batini et al (1992)., §6.4

Achieving Normalisation

One important quality is *normalisation*. This is one of the strengths by the *relational* data model, which we have already studied.



(a) The entity ORDER

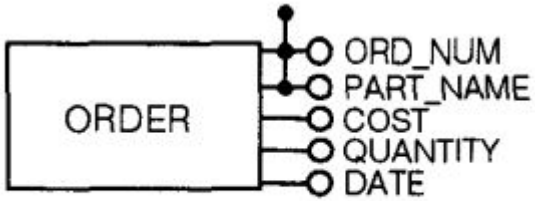
What problems can you foresee with the entity set to the left?

Time Allotted: 60 seconds

Source: Batini et al (1992)., §6.5

Achieving Normalisation

One important quality is *normalisation*. This is one of the strengths by the *relational* data model, the next unit of the course.



(a) The entity ORDER

What problems can you foresee with the entity set to the left?

Update Anomaly: what happens if we want to update a part name?

Insertion Anomaly: what happens if we insert a new order with the wrong part name?

Deletion Anomaly: what happens if we delete all orders for a specific part?

Time Allotted: 60 seconds

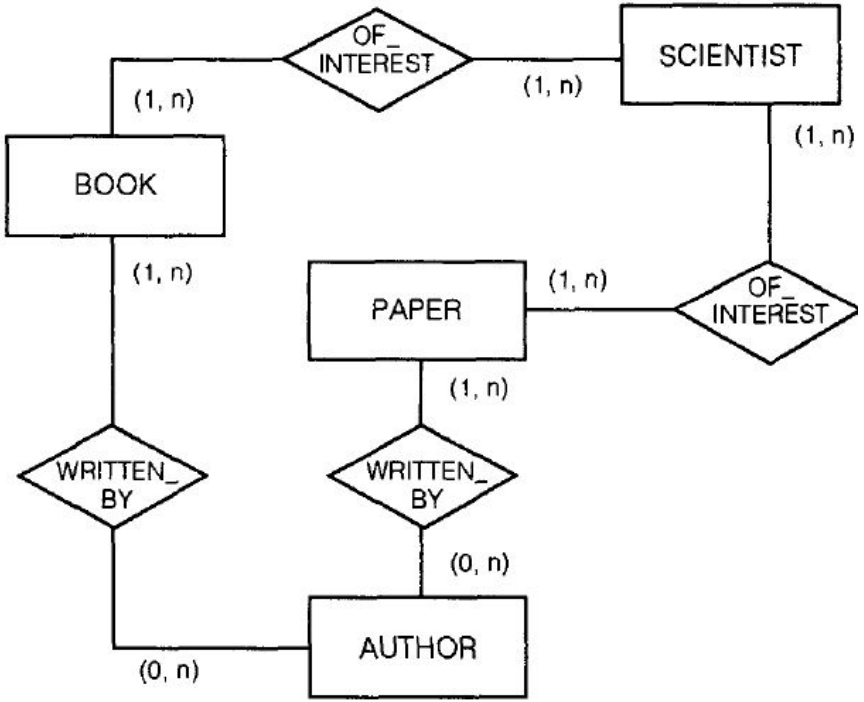
Source: Batini et al (1992)., §6.5

Summary / Exercise

Group Exercise

What transformations would you apply to improve this schema?

Why?



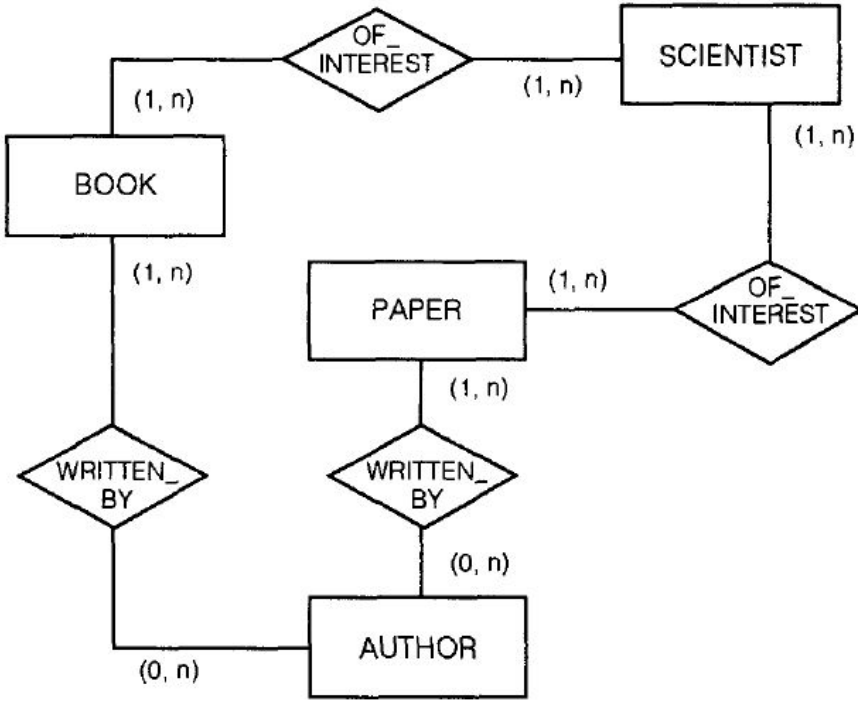
Time Allotted: 180 seconds

Source: Batini et al (1992)., §6.6

Group Exercise

Why?

- We can suspect there is some redundancy here (i.e., violation of "minimality"):
  - There are two relationships with the same name "of_interest"
  - There are another two relationships with the same name "written_by"
  - Books and papers seem similar
  - Scientists and authors seem similar and authors have (0,n) cardinalities

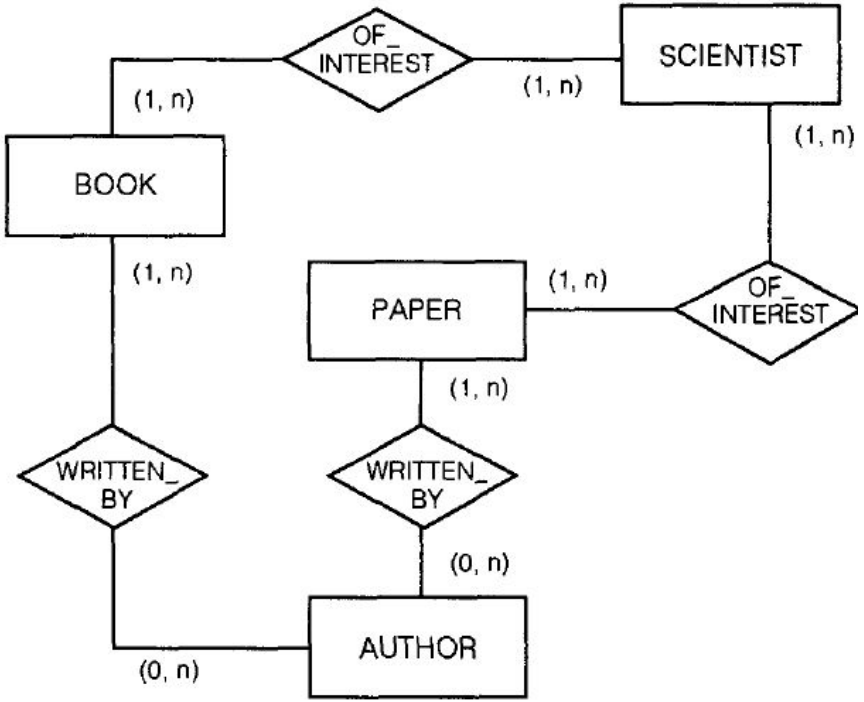


Source: Batini et al (1992)., §6.6

Group Exercise

Why?

- It lacks expressiveness, and self-explanation:
  - Book and paper seem to share the same relationships with scientist and author, but it's not clear if those are different
  - There is no way to express here that an author must be connected to at least one book *or* paper, but not necessarily one of each



Source: Batini et al (1992)., §6.6

Group Exercise

What transformations would you apply to improve this schema?

- Generalising book and paper to an “authored_work” improves all those aspects:
  - Minimal: we capture all requirements with a simpler schema
  - Expressive: there are no duplicate relationships
  - Complete: we can capture an additional constraint that authors might write *something*
  - Self-explanation: we do not need additional information to know that authors must have written something

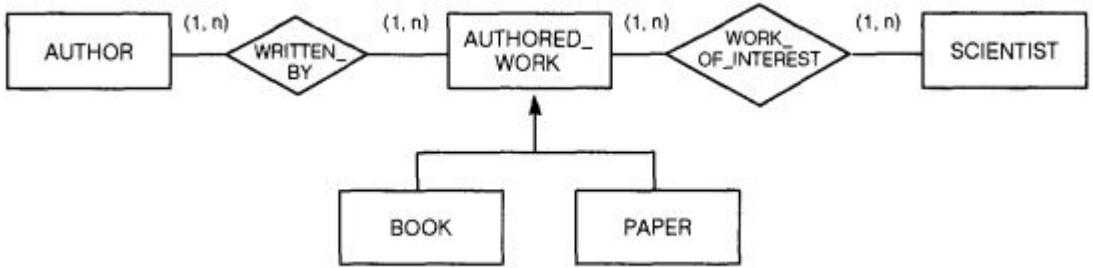


Figure 6.20 Improving expressiveness and self-explanation in the example

Source: Batini et al (1992)., §6.6